

MODEL AND RESPONSES

Model configuration

Learn about the Claude Code model configuration, including model aliases like `opusplan`

Available models

For the `model` setting in Claude Code, you can configure either:

- A **model alias**
- A **model name**
 - Anthropic API: a full model name
 - Bedrock: an inference profile ARN
 - Foundry: a deployment name
 - Vertex: a version name

ⓘ `ANTHROPIC_BASE_URL` changes where requests are sent, not which model answers them. To route Claude through an LLM gateway, see [LLM gateways](#).

Model aliases

Model aliases provide a convenient way to select model settings without remembering exact version numbers:

Model alias	Behavior
<code>default</code>	Special value that clears any model override and reverts to the recommended model for your account type. Not itself a model alias
<code>best</code>	Uses Fable 5 where your organization has access to it, otherwise the latest Opus model
<code>fable</code>	Uses Claude Fable 5 for your hardest and longest-running tasks
<code>sonnet</code>	Uses the latest Sonnet model for daily coding tasks
<code>opus</code>	Uses the latest Opus model for complex reasoning tasks

Model alias	Behavior
haiku	Uses the fast and efficient Haiku model for simple tasks
sonnet[1m]	Uses Sonnet with a 1 million token context window for long sessions
opus[1m]	Uses Opus with a 1 million token context window for long sessions
opusplan	Special mode that uses <code>opus</code> during plan mode, then switches to <code>sonnet</code> for execution

On the Anthropic API, `opus` resolves to Opus 4.8 and `sonnet` resolves to Sonnet 4.6. On Claude Platform on AWS, `opus` resolves to Opus 4.7 and `sonnet` resolves to Sonnet 4.6. On Bedrock, Vertex, and Foundry, `opus` resolves to Opus 4.6 and `sonnet` resolves to Sonnet 4.5; newer models are available on those providers by selecting the full model name explicitly or setting `ANTHROPIC_DEFAULT_OPUS_MODEL` or `ANTHROPIC_DEFAULT_SONNET_MODEL`.

Aliases point to the recommended version for your provider and update over time. To pin to a specific version, use the full model name, for example `claude-opus-4-8`, or set the corresponding environment variable like `ANTHROPIC_DEFAULT_OPUS_MODEL`.

 Opus 4.8 requires Claude Code v2.1.154 or later. Run `claude update` to upgrade.

Work with Fable 5

Claude Fable 5 is the most capable model in Claude Code, suited to tasks larger than a single sitting. It sustains long autonomous sessions, investigates before acting, and verifies its work more often than smaller models.

Fable 5 is not the default model. Select it with `/model fable`. Requests that its safety classifiers flag, most often in cybersecurity and biology domains, trigger automatic model fallback.

To get the most from Fable 5:

- **Describe the outcome, not the steps:** hand it the result you want and let it plan the path. To keep it working until that outcome holds, set a goal.
- **Hand it ambiguous problems:** root-cause investigations, outage debugging, and architecture decisions are where the extra investigation and verification pay off.
- **Skip the verification reminders:** it verifies its own work with less prompting, so reminders to test or check are usually unnecessary.
- **Size up larger tasks:** give it work you would normally break into pieces. It holds long sessions

without losing the thread.

❗ **Table 5** requires Claude Code v2.1.170 or later. Older versions do not show **Table 5** in the model picker and cannot select it. Run `claude update` to upgrade. **Table 5** is not available under zero data retention, where the `/model` picker either omits it or shows it disabled.

Setting your model

You can configure your model in several ways, listed in order of priority:

1. **During session:** use `/model <alias|name>` to switch immediately, or run `/model` with no argument to open the picker. The picker asks for confirmation when the conversation has prior output, since the next response re-reads the full history without cached context
2. **At startup:** launch with `claude --model <alias|name>`
3. **Environment variable:** set `ANTHROPIC_MODEL=<alias|name>`
4. **Settings:** configure permanently in your settings file using the `model` field

As of v2.1.153, `/model` saves your choice as the default for new sessions by writing the `model` field in your user settings. In the picker:

- `Enter` : switch model and save as your default
- `s` : switch model for this session only

Typing `/model <name>` directly behaves like `Enter`. Project and managed settings still take precedence and reapply on the next launch.

In v2.1.144 through v2.1.152, `/model` applied to the current session only and `d` in the picker saved a default.

The `--model` flag and `ANTHROPIC_MODEL` environment variable apply only to the session you launch with them. To run different models in different terminals at the same time, launch each one with its own `--model` flag rather than switching with `/model`.

Resumed sessions started with `claude --resume`, `--continue`, or the `/resume` picker keep the model they were using when the transcript was saved, regardless of the current `model` setting. If that model has been retired or is excluded by availableModels, the session falls through to the normal precedence order. This prevents another session's `/model` choice from changing the model on resume.

When the active model at startup comes from project or managed settings rather than your own selection, the startup header shows which settings file set it. Run `/model` to override; the project or managed setting reapplies on the next launch.

When the requested model has a scheduled retirement date or is automatically remapped to a newer version, Claude Code shows a warning that names the requested model. Interactive sessions show it as a startup notice. From v2.1.182, the same warning is written to stderr in **non-interactive mode** when using the default text output format. The check also covers a `model` set in **subagent frontmatter**. The stderr warning is suppressed for `--output-format json` and `stream-json`; read the actual model from the `modelUsage` field of the **result message** instead.

Example usage:

```
# Start with Opus
claude --model opus

# Switch to Sonnet during session
/model sonnet
```

Example settings file:

```
{
  "permissions": {
    ...
  },
  "model": "opus"
}
```

Restrict model selection

Enterprise administrators can use `availableModels` in **managed or policy settings** to restrict which models users can select. Entries match a model family such as `sonnet`, a version prefix such as `claude-sonnet-4-5`, or a full model ID such as `claude-sonnet-4-5-20250929`.

When `availableModels` is set, the allowlist applies everywhere a user can specify a model:

- **Main session model:** `/model`, the `--model` flag, the `ANTHROPIC_MODEL` environment variable, the `model` setting, and the model restored when **resuming a session**
- **Alias resolution:** the `ANTHROPIC_DEFAULT_OPUS_MODEL`, `ANTHROPIC_DEFAULT_SONNET_MODEL`,

`ANTHROPIC_DEFAULT_HAIKU_MODEL` , and `ANTHROPIC_DEFAULT_FABLE_MODEL` environment variables cannot redirect an allowed alias to a model outside the list

- **Fast mode:** `/fast` refuses to toggle when it would implicitly switch to an Opus model outside the list, with the message “is not in your organization’s allowed models”
- **Subagent models:** the `model` field in subagent frontmatter, the Agent tool’s `model` parameter, the model picker in `/agents` , and `CLAUDE_CODE_SUBAGENT_MODEL`
- **Skill and command models:** the `model` frontmatter in skills and commands
- **Advisor model:** the configured advisorModel setting and the `--advisor` flag
- **Background agent model:** the model selected in the dispatch picker

Switching to a blocked model with `/model` is rejected with an error, while a blocked `--model` flag, `ANTHROPIC_MODEL` , or `model` setting value is replaced at startup with a warning naming both the requested and substituted models, and the session starts on the default model. A blocked subagent, skill, or command override falls back to the inherited or default model rather than failing the request; a blocked `advisorModel` setting disables the advisor for the session, while a blocked `--advisor` flag value exits with an error at launch. Excluded models are hidden from the `/model` picker.

Automatic model changes are checked the same way: elements of a fallback model chain outside the allowlist are dropped, a plan-mode upgrade such as opusplan to an excluded model is skipped so planning continues on the session’s model, and an automatic model fallback whose target is excluded does not run, so the flagged request ends with a refusal instead. Enabling fast mode is refused when the model the session would run on afterward is outside the allowlist.

```
{
  "availableModels": ["sonnet", "haiku"]
}
```

Surface coverage

Every surface enforces the allowlist it receives. Which delivery mechanism reaches each surface differs:

Delivery mechanism	CLI and IDE	Desktop local sessions	Web, mobile, and cloud sessions	Agent SDK and non-interactive	Co
<u>Server-managed settings</u> from the admin console	Enforced	Enforced	Enforced	Enforced	No

Delivery mechanism	CLI and IDE	Desktop local sessions	Web, mobile, and cloud sessions	Agent SDK and non-interactive	Co
<u>MDM or managed settings files</u>	Enforced	Enforced	Not delivered	Enforced	En de

- Cloud sessions, on **Claude Code on the web** or in the Desktop app, run on Anthropic-managed VMs: settings deployed to your device do not reach them, so deliver the allowlist through server-managed settings. A mid-session model switch in a cloud session is rejected when the requested model is excluded by the allowlist. Server-side rejection at session creation applies to **organization model restrictions**, not the `availableModels` settings key.
- Cowork, the agentic-work tab in the Claude Desktop app, is not a Claude Code surface and does not receive server-managed settings by design. A managed settings file applies to Cowork sessions when it is present where the session runs; remote Cowork sessions run on Anthropic-managed VMs, where a device-deployed file is not present.
- Sessions on **third-party providers** such as Bedrock, Vertex AI, Foundry, and **Claude Platform on AWS** do not receive server-managed settings, so deliver the allowlist through MDM or managed settings files there.
- Server-managed delivery also requires the session to authenticate with an organization login or a directly configured API key. Fleets that generate keys only through an `apiKeyHelper` script should deliver the allowlist through MDM or managed settings files.
- The Desktop Code tab also hosts **SSH sessions**, which read the managed settings file from the remote host they run on. See **Desktop managed settings**.
- The model pickers on claude.ai and in the Desktop app hide or grey out models excluded by your organization’s allowlist. The picker state is a convenience for users; enforcement happens in the session.

Default model behavior

The Default option in the model picker is not affected by `availableModels` unless `enforceAvailableModels` is also set. On its own, `availableModels` leaves Default available, resolving to the system’s runtime default **based on the user’s subscription tier**. If the tier default is a model you intend to restrict, set `enforceAvailableModels` as well.

An empty `availableModels` array never engages the Default-model enforcement: with `availableModels: []`, named model selections are blocked but the Default model for the account type remains usable regardless of `enforceAvailableModels`.

Enforce the allowlist for the Default model

Set `enforceAvailableModels: true` alongside a non-empty `availableModels` in managed settings to extend the allowlist to the Default option. This requires Claude Code v2.1.175 or later.

```
{
  "availableModels": ["sonnet", "haiku"],
  "enforceAvailableModels": true
}
```

When the default model for the user's account type is not in the allowlist, the Default option instead resolves to the first `availableModels` entry that names an allowed, available model, and the `/model` picker's Default row shows that model. This applies everywhere the default is reached: session startup, selecting Default in `/model`, the `"default"` keyword in **fallback model chains**, and the fallback used when an excluded selection is dropped.

`enforceAvailableModels` has no effect when `availableModels` is unset or empty: with `availableModels: []`, the Default model for the account type remains usable, so the setting cannot lock users out of every model. When `availableModels` is non-empty but no entry resolves to an allowed and available model, enforcement is skipped and Default resolves to the account-type default, with a warning visible only under `--debug`. Keep at least one guaranteed-available entry in the list to avoid this.

Deploy both keys in the **highest-precedence managed source**: admin-deployed managed sources do not merge, so a pair placed in a managed settings file is ignored when the admin console delivers any settings.

Control the model users run on

The `model` setting is an initial selection, not enforcement. It sets which model is active when a session starts, but users can still open `/model` and pick Default, which resolves to the system default for their tier regardless of what `model` is set to, unless **enforceAvailableModels** redirects it.

To fully control the model experience, combine these settings:

- `availableModels` : restricts which named models users can switch to
- `enforceAvailableModels` : extends the `availableModels` allowlist to the Default option, so Default cannot resolve to a model outside the list

- `model` : sets the initial model selection when a session starts
- `ANTHROPIC_DEFAULT_SONNET_MODEL` / `ANTHROPIC_DEFAULT_OPUS_MODEL` / `ANTHROPIC_DEFAULT_HAIKU_MODEL` / `ANTHROPIC_DEFAULT_FABLE_MODEL` : control what the Default option and the `sonnet` , `opus` , `haiku` , and `fable` aliases resolve to

This example starts users on Sonnet 4.5, limits the picker to Sonnet and Haiku, and ensures Default resolves to a model on the allowlist rather than the tier default:

```
{
  "model": "claude-sonnet-4-5",
  "availableModels": ["claude-sonnet-4-5", "haiku"],
  "enforceAvailableModels": true,
  "env": {
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "claude-sonnet-4-5"
  }
}
```

Without `enforceAvailableModels` or the `env` block, a user who selects Default in the picker would get the latest release for their tier, bypassing the version pin in `model` and `availableModels` . The two settings cover different scopes: `enforceAvailableModels` makes Default obey the allowlist, while the `env` block pins which version a permitted alias such as `sonnet` resolves to. Use `enforceAvailableModels` alone when restricting model families is enough; add the `env` block when you also need to pin a specific version.

Merge behavior

When the **highest-precedence managed settings source** defines `availableModels` , that list alone applies: entries in user, project, or local settings cannot extend it, and admin-deployed managed sources do not merge with each other, so a list deployed in a managed settings file is ignored when server-managed settings deliver any keys. Otherwise, lists from user, project, and local settings are **concatenated and deduplicated** like other array settings. As of Claude Code v2.1.175, the managed list replaces lower-precedence entries; earlier versions merge them.

Within the effective list, an entry naming a specific model in a family, whether a version prefix or a full model ID, disables that family's wildcard entry: `["sonnet", "claude-sonnet-4-5"]` allows only Sonnet 4.5 versions, not every Sonnet model.

Mantle model IDs

When the **Bedrock Mantle endpoint** is enabled, entries in `availableModels` that start with `anthropic.` are added to the `/model` picker as custom options and routed to the Mantle endpoint. This is an exception to the alias matching described in **Pin models for third-party deployments**. The setting still restricts the picker to listed entries, and a Mantle ID embeds a family name, so it counts as a specific entry and disables that family's wildcard: alongside any Mantle IDs, list the version prefixes or full IDs you want to keep selectable. See **Merge behavior**.

Organization model restrictions

Organization admins restrict which models members can run by disabling individual models in the Claude Console. Use this Console toggle instead of `availableModels` when your members authenticate through the Anthropic API and you want one org-wide switch without deploying settings files. This restriction is delivered with the account's entitlements when Claude Code authenticates, separate from any `availableModels` list in settings, and the server enforces the same restriction independently when a session is created. Requires Claude Code v2.1.187 or later.

A restricted model is hidden from the `/model` picker. Selecting it by name with `--model`, the `ANTHROPIC_MODEL` environment variable, or the `model` setting shows the notice `Model "<name>" is restricted by your organization's settings. Using <model> instead.` and the session starts on an allowed model. Typing `/model <name>` for a restricted model is rejected with `Model '<name>' is restricted by your organization's settings. Run /model to choose a different model.` and the session keeps its current model.

Both restrictions apply together: a model is selectable only when it is permitted by `availableModels` and not restricted by the organization. Organization restrictions are delivered to sessions on the Anthropic API and **LLM gateway** deployments. Sessions on Bedrock, Vertex AI, Foundry, and Claude Platform on AWS do not receive them, so use `availableModels` on those providers instead.

Special model behavior

`default` model setting

The behavior of `default` depends on your account type:

- **Max, Team Premium, Enterprise pay-as-you-go, and Anthropic API:** defaults to Opus 4.8
- **Claude Platform on AWS:** defaults to Opus 4.7

- **Pro, Team Standard, and Enterprise subscription seats:** defaults to Sonnet 4.6
- **Bedrock, Vertex, and Foundry:** defaults to Sonnet 4.5

Enterprise pay-as-you-go means an Enterprise organization billed by usage rather than by subscription seat.

When managed settings **enforce the allowlist for the Default model** and the account-type default is not in `availableModels` , `default` resolves to the enforced Default instead of the account-type default above.

Fable 5 is not the default model on any account type. Sessions use Fable 5 only after you choose it, with `/model fable` , a `model` setting, or the `best` alias where Fable 5 is available. Choosing it with `/model` saves it as the selected model in your user settings, so later sessions start on Fable 5 until you change models.

`opusplan` model setting

The `opusplan` model alias provides an automated hybrid approach:

- **In plan mode:** uses `opus` for complex reasoning and architecture decisions
- **In execution mode:** automatically switches to `sonnet` for code generation and implementation

This pairs Opus’s reasoning for planning with Sonnet’s efficiency for execution.

The plan-mode Opus phase uses the same context window as the `opus` model setting. On subscription tiers where Opus is **automatically upgraded to 1M context**, `opusplan` receives the upgrade in plan mode as well. To force 1M context for both phases when you are not on an auto-upgrade tier, set the model to `opusplan[1m]` .

When **availableModels** excludes Opus, `opusplan` stays on Sonnet in plan mode instead of switching. Similarly, a Haiku session that would normally upgrade to Sonnet in plan mode stays on Haiku when Sonnet is excluded.

For a hybrid approach where Claude decides mid-task when to consult a second model rather than switching at the plan boundary, see the **advisor tool**.

Fallback model chains

When the primary model is overloaded, unavailable, or returns another non-retryable server error, Claude Code can switch to a fallback model instead of failing the request. Authentication, billing,

rate-limit, request-size, and transport errors never trigger a switch; those follow their normal retry and error handling.

Configure one or more fallback models and Claude Code tries them in order, showing a notice when it switches. The switch lasts for the current turn only, so your next message tries the primary model first again. Chains are capped at three models after duplicate removal, and extra entries are ignored.

Set a chain for one session with the `--fallback-model` flag, which accepts a comma-separated list:

```
claude --fallback-model sonnet,haiku
```

To persist a chain across sessions, set `fallbackModel` in settings as an array:

```
{
  "fallbackModel": ["claude-sonnet-4-6", "claude-haiku-4-5"]
}
```

The `--fallback-model` flag takes precedence over the `fallbackModel` setting. Each element accepts a model name or alias, and `"default"` expands to the default model.

Two cases cause an element to be skipped:

- **Unavailable model:** a model that can't be reached, such as a retired model pinned in settings, is skipped and Claude Code continues to the next element.
- **Outside the allowlist:** an element not permitted by availableModels is dropped when the chain is read and never tried.

Automatic model fallback

This section covers content-based fallback from Fable 5. For availability-based fallback when a model is overloaded or unavailable, see Fallback model chains.

Fable 5 runs with safety classifiers for cybersecurity and biology content. When a classifier flags a request, Claude Code re-runs that request on the default Opus model and shows a notice in the transcript: Opus 4.8 on the Anthropic API and LLM gateway deployments, or Opus 4.7 on Claude Platform on AWS.

The session then continues on that Opus model. To return to Fable 5, run `/model fable`.

The fallback target is checked against `availableModels`. When it is blocked, no fallback occurs. The refusal is shown as a normal error and the session's model is unchanged.

Check what triggered fallback

Fallback can trigger on the first request of a session, before you send anything unusual, because the first request carries workspace context such as your CLAUDE.md content and git status. A repository that contains security or biology material can trip the classifier on that context alone.

To check whether customizations are the trigger, start a session with `claude --safe-mode`, which disables customizations such as CLAUDE.md, skills, MCP servers, and hooks. Git status and directory names are not customizations and are still included.

Ask before switching

To decide what happens each time a request is flagged, rather than switching automatically, run `/config` and turn off “switch models when a message is flagged”. A flagged request then pauses the session with two options: switch to the Opus model, or edit the prompt and retry on Fable 5.

Some cases behave differently:

- If both models flag the same request, you can edit the prompt and retry, or start a new session.
- On mobile **Claude Code on the web** sessions, editing and retrying is not supported. Switch models, or continue the session from a desktop browser or the desktop app.
- In **non-interactive mode** and SDK integrations that can't show the prompt, a flagged request ends the turn with a refusal instead.
- When the fallback target is blocked by `availableModels`, the prompt is not shown. The flagged request ends with the refusal, the same as automatic fallback when the target is blocked.

Enable fallback on Bedrock, Vertex AI, and Foundry

On **Amazon Bedrock**, **Google Vertex AI**, and **Microsoft Foundry**, model IDs are provider-specific, so automatic fallback only operates when Claude Code can identify both models involved:

- Claude Code must recognize the current model as Fable 5: the model ID contains `claude-fable-5`, matches the value of `ANTHROPIC_DEFAULT_FABLE_MODEL`, or is mapped with `modelOverrides`.
- The fallback target must resolve to an Opus model: the value of `ANTHROPIC_DEFAULT_OPUS_MODEL` if set, otherwise an Opus 4.8 entry in the provider's model list.

If either model can't be identified, Claude Code does not switch automatically. The flagged request ends with a refusal message, and you can switch models with `/model` and retry. To enable automatic fallback on these providers, set `ANTHROPIC_DEFAULT_FABLE_MODEL` to your Fable 5 model ID and `ANTHROPIC_DEFAULT_OPUS_MODEL` to your Opus 4.8 model ID.

Security research and biology workloads

Workloads in offensive security or biology, including penetration testing, Capture the Flag (CTF) exercises, and biology-adjacent codebases, trigger fallback frequently, often on the first request. For substantive biology work, expect nearly all requests to reroute.

This is expected routing for these domains, not an account flag. If your organization needs Fable-class capability for this work, ask your Anthropic account team about trusted access programs.

Adjust effort level

Effort levels control adaptive reasoning, which lets the model decide whether and how much to think on each step based on task complexity. Lower effort is faster and cheaper for straightforward tasks, while higher effort provides deeper reasoning for complex problems.

The available effort levels depend on the model. Models not listed here do not support effort:

Model	Levels
Fable 5	<code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code>
Opus 4.8 and Opus 4.7	<code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code>
Opus 4.6 and Sonnet 4.6	<code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code>

If you set a level the active model does not support, Claude Code falls back to the highest supported level at or below the one you set. For example, `xhigh` runs as `high` on Opus 4.6.

The default effort is `high` on Fable 5, Opus 4.8, Opus 4.6, and Sonnet 4.6, and `xhigh` on Opus 4.7.

When you first run Fable 5, Opus 4.8, or Opus 4.7, Claude Code applies that model's default effort even if you previously set a different level for another model: `high` on Fable 5 and Opus 4.8, and `xhigh` on Opus 4.7. Run `/effort` again to choose a different level after switching.

`low` , `medium` , `high` , and `xhigh` persist across sessions. `max` provides the deepest reasoning with no constraint on token spending and applies to the current session only, except when set

through the `CLAUDE_CODE_EFFORT_LEVEL` environment variable.

The `/effort` menu also offers `ultracode`. Ultracode is a Claude Code setting rather than a model effort level: it sends `xhigh` to the model and additionally has Claude orchestrate **dynamic workflows** for substantive tasks. It applies to the current session only. Set it through `/effort`, or pass `"ultracode": true` via `--settings` or an Agent SDK control request. It is not part of the `effortLevel` setting, the `--effort` flag, or `CLAUDE_CODE_EFFORT_LEVEL`.

Choose an effort level

Each level trades token spend against capability. The default suits most coding tasks; adjust when you want a different balance.

Level	When to use it
<code>low</code>	Reserve for short, scoped, latency-sensitive tasks that are not intelligence-sensitive
<code>medium</code>	Reduces token usage for cost-sensitive work that can trade off some intelligence
<code>high</code>	Balances token usage and intelligence. Default on Fable 5, Opus 4.8, Opus 4.6, and Sonnet 4.6
<code>xhigh</code>	Deeper reasoning at higher token spend. Default on Opus 4.7
<code>max</code>	Can improve performance on demanding tasks but may show diminishing returns and is prone to overthinking. Test before adopting broadly
<code>ultracode</code>	A Claude Code setting that plans a <u>dynamic workflow</u> for each substantive task with <code>xhigh</code> per-message reasoning. Session-only

The effort scale is calibrated per model, so the same level name does not represent the same underlying value across models.

Use ultrathink for one-off deep reasoning

Include `ultrathink` anywhere in your prompt to request deeper reasoning on that turn without changing your session effort setting. Claude Code recognizes the keyword and adds an in-context instruction. The effort level sent to the API is unchanged. Other phrases such as “think”, “think hard”, and “think more” are passed through as ordinary prompt text and are not recognized as keywords.

Set the effort level

You can change effort through any of the following:

- `/effort :run /effort` with no arguments to open an interactive slider, `/effort` followed by

a level name to set it directly, or `/effort auto` to reset to the model default

- In `/model` : use left/right arrow keys to adjust the effort slider when selecting a model
- `--effort` **flag**: pass a level name to set it for a single session when launching Claude Code
- **Environment variable**: set `CLAUDE_CODE_EFFORT_LEVEL` to a level name or `auto`
- **Settings**: set `effortLevel` to `low`, `medium`, `high`, or `xhigh` in your settings file. `max` and `ultracode` are **session-only** and are not accepted here
- **Skill and subagent frontmatter**: set `effort` in a **skill** or **subagent** markdown file to override the effort level when that skill or subagent runs

The environment variable takes precedence over all other methods, then your configured level, then the model default. Frontmatter effort applies when that skill or subagent is active, overriding the session level but not the environment variable.

The effort slider appears in `/model` when a supported model is selected. The current effort level is also displayed next to the logo and spinner, for example “with low effort”, so you can confirm which setting is active without opening `/model` .

Adaptive reasoning and fixed thinking budgets

Adaptive reasoning makes thinking optional on each step, so Claude can respond faster to routine prompts and reserve deeper thinking for steps that benefit from it. If you want Claude to think more or less often than the current level produces, you can say so directly in your prompt or in `CLAUDE.md` ; the model responds to that guidance within its effort setting.

Opus 4.7 and later always use adaptive reasoning, as does Fable 5. The fixed thinking budget mode and `CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING` do not apply to them.

On Opus 4.6 and Sonnet 4.6, you can set `CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING=1` to revert to the previous fixed thinking budget controlled by `MAX_THINKING_TOKENS` . See **environment variables**.

Extended thinking

Extended thinking is the reasoning Claude emits before responding. On models that support **adaptive reasoning**, the effort level is the primary control for how much thinking happens; the settings below turn thinking on or off and control how it displays.

Control	How to set it
Toggle for the current session	Press <code>Option+T</code> on macOS or <code>Alt+T</code> on Windows and Linux
Set the global default	Run <code>/config</code> and toggle thinking mode. Saved as <code>alwaysThinkingEnabled</code> in <code>~/.claude/settings.json</code>
Disable regardless of effort	Set <code>MAX_THINKING_TOKENS=0</code> , which turns thinking off on the Anthropic API except on Fable 5. On third-party providers this omits the <code>thinking</code> parameter instead, and adaptive-reasoning models may still think. Other values apply only with a <u>fixed thinking budget</u>

Thinking cannot be turned off on Fable 5. The session toggle, `alwaysThinkingEnabled`, and `MAX_THINKING_TOKENS=0` have no effect there, and Fable 5 decides per step how much to think based on the effort level.

Thinking output is collapsed by default. Press `Ctrl+0` to toggle verbose mode and see the reasoning as gray italic text. Interactive sessions on the Anthropic API receive redacted thinking blocks by default, so set `showThinkingSummaries: true` in **settings** if you want the full summaries available when you expand. You are charged for all thinking tokens generated, even when collapsed or redacted.

Extended context

Fable 5, Opus 4.6 and later, and Sonnet 4.6 support a **1 million token context window** for long sessions with large codebases.

Availability varies by model and plan. On Max, Team, and Enterprise plans, Opus is automatically upgraded to 1M context with no additional configuration. This applies to both Team Standard and Team Premium seats. On the Anthropic API, Fable 5, Opus 4.8, and Opus 4.7 always run with the 1M window. Sonnet with 1M context is not part of the automatic upgrade and requires **usage credits** on every subscription plan, including Max.

Plan	Opus with 1M context	Sonnet with 1M context
Max, Team, and Enterprise	Included with subscription	Requires <u>usage credits</u>
Pro	Requires <u>usage credits</u>	Requires <u>usage credits</u>
API and pay-as-you-go	Full access	Full access

To disable 1M context entirely, set `CLAUDE_CODE_DISABLE_1M_CONTEXT=1`. This removes 1M model

variants from the model picker. See [environment variables](#).

The 1M context window uses standard model pricing with no premium for tokens beyond 200K. For plans where extended context is included with your subscription, usage remains covered by your subscription. For plans that access extended context through usage credits, tokens are billed to usage credits.

If your account supports 1M context, the option appears in the `/model` picker in the latest versions of Claude Code. If you don't see it, try restarting your session.

You can also use the `[1m]` suffix with model aliases or full model names:

```
# Use the opus[1m] or sonnet[1m] alias
/model opus[1m]
/model sonnet[1m]

# Or append [1m] to a full model name
/model claude-opus-4-8[1m]
```

Checking your current model

You can see which model you're currently using in two places:

- In the [status line](#), if you have one configured
- In `/status`, which also displays your account information

Add a custom model option

Use `ANTHROPIC_CUSTOM_MODEL_OPTION` to add a single custom entry to the `/model` picker without replacing the built-in aliases. This is useful for testing model IDs that Claude Code does not list by default. For LLM gateway deployments, Claude Code can populate the picker from the gateway's `/v1/models` endpoint when `CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY=1` is set, so this variable is needed only when discovery is disabled or does not return the model you want. See [gateway model discovery](#).

This example sets all three variables to make a gateway-routed Opus deployment selectable:

```
export ANTHROPIC_CUSTOM_MODEL_OPTION="my-gateway/claude-opus-4-8"
export ANTHROPIC_CUSTOM_MODEL_OPTION_NAME="Opus via Gateway"
export ANTHROPIC_CUSTOM_MODEL_OPTION_DESCRIPTION="Custom
deployment routed through the internal LLM gateway"
```

The custom entry appears at the bottom of the `/model` picker.

`ANTHROPIC_CUSTOM_MODEL_OPTION_NAME` and `ANTHROPIC_CUSTOM_MODEL_OPTION_DESCRIPTION` are optional. If omitted, the model ID is used as the name and the description defaults to `Custom model (<model-id>)`.

Claude Code skips validation for the model ID set in `ANTHROPIC_CUSTOM_MODEL_OPTION`, so you can use any string your API endpoint accepts. When `availableModels` is set, include the custom model ID in the allowlist as well: the custom entry is filtered from the picker and a `--model` selection of it is rejected like any other excluded model. A custom ID that embeds a family name, such as `my-gateway/claude-opus-4-8`, counts as a specific entry for that family and disables its wildcard, so also list the versions you intend to keep selectable. See [Merge behavior](#).

Environment variables

You can use the following environment variables to control the model names that the aliases map to. Each value must be a full model name, or the equivalent identifier for your API provider.

Environment variable	Description
<code>ANTHROPIC_DEFAULT_FABLE_MODEL</code>	The model to use for <code>fable</code> , and the model ID Claude Code recognizes as Fable 5 for <u>automatic model fallback</u> on third-party providers
<code>ANTHROPIC_DEFAULT_OPUS_MODEL</code>	The model to use for <code>opus</code> , or for <code>opusplan</code> when Plan Mode is active.
<code>ANTHROPIC_DEFAULT_SONNET_MODEL</code>	The model to use for <code>sonnet</code> , or for <code>opusplan</code> when Plan Mode is not active.
<code>ANTHROPIC_DEFAULT_HAIKU_MODEL</code>	The model to use for <code>haiku</code> , or <u>background functionality</u>
<code>CLAUDE_CODE_SUBAGENT_MODEL</code>	The model to use for all <u>subagents</u> and <u>agent teams</u> . Overrides the per-invocation <code>model</code> parameter and the subagent definition's <code>model</code> frontmatter. Set to <code>inherit</code> to use normal model resolution instead

Note: `ANTHROPIC_SMALL_FAST_MODEL` is deprecated in favor of `ANTHROPIC_DEFAULT_HAIKU_MODEL`.

Pin models for third-party deployments

When deploying Claude Code through Bedrock, Vertex AI, Foundry, or Claude Platform on AWS, pin model versions before rolling out to users.

Without pinning, Claude Code uses model aliases such as `fable`, `opus`, `sonnet`, and `haiku` that resolve to a built-in default model ID for each provider. That default can lag the newest Anthropic release, and the model it points to may not yet be enabled in a user's account. When the default is unavailable, Bedrock and Vertex AI users see a notice and fall back to the previous version for that session, while Foundry users see errors because Foundry has no equivalent startup check.

⚠ Set the model environment variables to specific version IDs as part of your initial setup. Pinning lets you control when your users move to a new model.

Use the following environment variables with version-specific model IDs for your provider:

Provider	Example
Bedrock	<code>export ANTHROPIC_DEFAULT_OPUS_MODEL='us.anthropic.claude-opus-4-8'</code>
Vertex AI	<code>export ANTHROPIC_DEFAULT_OPUS_MODEL='claude-opus-4-8'</code>
Foundry	<code>export ANTHROPIC_DEFAULT_OPUS_MODEL='claude-opus-4-8'</code>

Apply the same pattern for `ANTHROPIC_DEFAULT_FABLE_MODEL`, `ANTHROPIC_DEFAULT_SONNET_MODEL`, and `ANTHROPIC_DEFAULT_HAIKU_MODEL`. For current and legacy model IDs across all providers, see Models overview. To upgrade users to a new model version, update these environment variables and redeploy.

To enable extended context for a pinned model, append `[1m]` to the model ID in `ANTHROPIC_DEFAULT_OPUS_MODEL` or `ANTHROPIC_DEFAULT_SONNET_MODEL`:

```
export ANTHROPIC_DEFAULT_OPUS_MODEL='claude-opus-4-8[1m]'
```

The `[1m]` suffix applies the 1M context window to all usage of the `opus` and `sonnet` aliases, including the plan-mode Opus phase of opusplan.

- Claude Code strips the suffix before sending the model ID to your provider.
- Only append `[1m]` when the underlying model supports 1M context.
- The suffix is read per variable, not per model. On Bedrock, Vertex, and Foundry, a model ID

without `[1m]` in one variable uses 200K context even if another variable sets the same model with the suffix.

❗ An `availableModels` allowlist delivered through MDM or a managed settings file still applies when using third-party providers; **server-managed settings are not delivered there**. Filtering matches on a model alias such as `opus` , a version prefix such as `claude-opus-4-8` , or the full provider-form model ID. Provider-specific prefixes such as `us.anthropic.` are not stripped, so to allow a specific model, list the same provider-form ID the picker shows, or map it through `modelOverrides` . Any `[1m]` suffix is stripped from both the allowlist entry and the requested model before matching.

Customize pinned model display and capabilities

When you pin a model on a third-party provider, the provider-specific ID appears as-is in the `/model` picker and Claude Code may not recognize which features the model supports. You can override the display name and declare capabilities with companion environment variables for each pinned model.

These variables take effect on third-party providers such as Bedrock, Vertex AI, and Foundry. The `_NAME` and `_DESCRIPTION` variables also take effect when `ANTHROPIC_BASE_URL` points to an LLM gateway. They have no effect when connecting directly to `api.anthropic.com` .

Environment variable	Description
<code>ANTHROPIC_DEFAULT_OPUS_MODEL_NAME</code>	Display name for the pinned Opus model in the <code>/model</code> picker. Defaults to the model ID when not set
<code>ANTHROPIC_DEFAULT_OPUS_MODEL_DESCRIPTION</code>	Display description for the pinned Opus model in the <code>/model</code> picker. Defaults to <code>Custom Opus model</code> when not set
<code>ANTHROPIC_DEFAULT_OPUS_MODEL_SUPPORTED_CAPABILITIES</code>	Comma-separated list of capabilities the pinned Opus model supports

The same `_NAME` , `_DESCRIPTION` , and `_SUPPORTED_CAPABILITIES` suffixes are available for `ANTHROPIC_DEFAULT_SONNET_MODEL` , `ANTHROPIC_DEFAULT_HAIKU_MODEL` , `ANTHROPIC_DEFAULT_FABLE_MODEL` , and `ANTHROPIC_CUSTOM_MODEL_OPTION` .

Claude Code enables features like effort levels and extended thinking by matching the model ID against known patterns. Provider-specific IDs such as Bedrock ARNs or custom deployment names often don't match these patterns, leaving supported features disabled. Set `_SUPPORTED_CAPABILITIES` to tell Claude Code which features the model actually supports:

Capability value	Enables
effort	<u>Effort levels</u> and the <code>/effort</code> command
xhigh_effort	The <code>xhigh</code> effort level
max_effort	The <code>max</code> effort level
thinking	<u>Extended thinking</u>
adaptive_thinking	Adaptive reasoning that dynamically allocates thinking based on task complexity
interleaved_thinking	Thinking between tool calls

When `_SUPPORTED_CAPABILITIES` is set, listed capabilities are enabled and unlisted capabilities are disabled for the matching pinned model. When the variable is unset, Claude Code falls back to built-in detection based on the model ID.

This example pins Opus to a Bedrock custom model ARN, sets a friendly name, and declares its capabilities:

```
export ANTHROPIC_DEFAULT_OPUS_MODEL='arn:aws:bedrock:us-east-1:123456789012:custom-model/abc'
export ANTHROPIC_DEFAULT_OPUS_MODEL_NAME='Opus via Bedrock'
export ANTHROPIC_DEFAULT_OPUS_MODEL_DESCRIPTION='Opus 4.7 routed through a Bedrock custom endpoint'
export ANTHROPIC_DEFAULT_OPUS_MODEL_SUPPORTED_CAPABILITIES='effort,xhigh_effort,max_effort,thinking,adaptive_thinking,interleaved_thinking'
```

Override model IDs per version

The family-level environment variables above configure one model ID per family alias. If you need to map several versions within the same family to distinct provider IDs, use the `model0verrides` setting instead.

`model0verrides` maps individual Anthropic model IDs to the provider-specific strings that Claude Code sends to your provider’s API. When a user selects a mapped model in the `/model` picker, Claude Code uses your configured value instead of the built-in default.

This lets enterprise administrators route each model version to a specific Bedrock inference profile ARN, Vertex AI version name, or Foundry deployment name for governance, cost allocation, or

regional routing.

Set `modelOverrides` in your settings file:

```
{
  "modelOverrides": {
    "claude-opus-4-7": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/opus-prod",
    "claude-opus-4-6": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/opus-46-prod",
    "claude-sonnet-4-6": "arn:aws:bedrock:us-east-2:123456789012:application-inference-profile/sonnet-prod"
  }
}
```

Keys must be Anthropic model IDs as listed in the [Models overview](#). For dated model IDs, include the date suffix exactly as it appears there. Unknown keys are ignored.

Overrides replace the built-in model IDs that back each entry in the `/model` picker. On Bedrock, overrides take precedence over any inference profiles that Claude Code discovers automatically at startup. Values you supply directly through `ANTHROPIC_MODEL`, `--model`, or the `ANTHROPIC_DEFAULT*_MODEL` environment variables are passed to the provider as-is and are not transformed by `modelOverrides`.

`modelOverrides` works alongside `availableModels`. The allowlist is evaluated against the Anthropic model ID, not the override value, so an entry like `"opus"` in `availableModels` continues to match even when Opus versions are mapped to ARNs. When `enforceAvailableModels` is set in managed settings, the enforced Default resolves through `modelOverrides` from the **highest-precedence managed source** only. An admin's mapping, such as a version pinned to an inference profile ARN, is honored in the enforced Default. Overrides from user or project settings do not affect it.

Prompt caching configuration

Claude Code automatically uses **prompt caching** to optimize performance and reduce costs. You can disable prompt caching globally or for specific model tiers:

Environment variable	Description
DISABLE_PROMPT_CACHING	Set to 1 to disable prompt caching for all models. Takes precedence over the per-model settings
DISABLE_PROMPT_CACHING_HAIKU U	Set to 1 to disable prompt caching for Haiku models only
DISABLE_PROMPT_CACHING_SONNET ET	Set to 1 to disable prompt caching for Sonnet models only
DISABLE_PROMPT_CACHING_OPUS S	Set to 1 to disable prompt caching for Opus models only
DISABLE_PROMPT_CACHING_FABLE E	Set to 1 to disable prompt caching for Fable models only

To change the cache TTL or learn what triggers a cache miss, see [How Claude Code uses prompt caching](#).